Assignment No. 02

Title: Construct an expression tree from postfix expression and perform recursive and non- recursive traversals.

Introduction:

Expression Tree:

An expression tree is a binary tree in which each internal node corresponds to the operator and each leaf node corresponds to the operand.

- Leaf nodes → Operands (variables or constants)
- Internal nodes → Operators (+, −, ×, /, etc.)

It is mainly used in:

- Compiler design
- Expression evaluation
- Parsing arithmetic expressions

Structure of Expression Tree:

- It is a binary tree.
- Every operator has
    1. Left Child → Left Operand
    2. Right Child → Right Operand
- Operands are always leaf nodes.

Expression trees can be constructed from:

1. Infix notation → a + b
2. Prefix notation → + a b
3. Postfix notation → a b +

Operator Precedence Maintained in Expression Tree

From highest to lowest:

1. () → Parentheses
2. ^ → Exponentiation
3. * / % → Multiplication, Division, Modulus
4. + - → Addition, Subtraction

Pseudocodes:

1. Create Node

```
FUNCTION create(data)
   CREATE new node called root
   SET root.data = data
   SET root.l = NULL
   SET root.r = NULL
   RETURN root
END FUNCTION
```

2. Inorder Recursive Traversal

```
FUNCTION inorder(root)
   IF root is not NULL THEN
      CALL inorder(root.l)
      PRINT root.data
      CALL inorder(root.r)
   END IF
END FUNCTION
```

3. Preorder Recursive Traversal

```
FUNCTION preorder(root)
   IF root is not NULL THEN
      PRINT root.data
      CALL preorder(root.l)
      CALL preorder(root.r)
   END IF
END FUNCTION
```

4. Postorder Recursive Traversal

```
FUNCTION postorder(root)
   IF root is not NULL THEN
      CALL postorder(root.l)
      CALL postorder(root.r)
      PRINT root.data
   END IF
END FUNCTION
```

5. Construct Tree

```
FUNCTION constructTree(exp)

   CREATE empty stack st

   FOR each character ch in exp

      IF ch is operand (letter or digit) THEN
         CREATE node using create(ch)
         PUSH node into stack

      ELSE   // ch is operator

         CREATE node using create(ch)

         SET node.r = TOP element of stack
         POP stack

         SET node.l = TOP element of stack
         POP stack

         PUSH node into stack
```

```
        END IF

    END FOR

    RETURN TOP element of stack

END FUNCTION
```

6. Inorder Non-Recursive Traversal

```
FUNCTION inorderNR(root)

    CREATE empty stack st
    SET current = root

    WHILE current is not NULL OR stack is not empty DO

        WHILE current is not NULL DO
            PUSH current into stack
            SET current = current.l
        END WHILE

        SET temp = TOP element of stack
        POP stack

        PRINT temp.data

        SET current = temp.r

    END WHILE

END FUNCTION
```

7. Preorder Non-Recursive Traversal

```
FUNCTION preorderNR(root)

    CREATE empty stack st
    PUSH root into stack

    WHILE stack is not empty DO

        SET temp = TOP element of stack
        POP stack

        PRINT temp.data

        IF temp.r is not NULL THEN
            PUSH temp.r into stack
        END IF

        IF temp.l is not NULL THEN
```

```
        PUSH temp.l into stack
      END IF

    END WHILE

END FUNCTION
```

Algorithms:

1. Create Node

Step 1: Start
Step 2: Allocate memory for a new node called root
Step 3: Set root.data = data
Step 4: Set root.l = NULL
Step 5: Set root.r = NULL
Step 6: Return root
Step 7: Stop

2. Construct Tree

Step 1: Start
Step 2: Create an empty stack st
Step 3: For each character ch in expression exp, repeat Steps 4–10
Step 4: If ch is an operand (letter or digit), then follow step 6
Step 5: Create a node using CREATE_NODE(ch)
Step 6: Push the node into stack
Step 7: Else (ch is an operator) follow step 8 to 11
Step 8: Create a node using CREATE_NODE(ch)
Step 9: Pop top element of stack and assign it to node.r
Step 10: Pop next top element and assign it to node.l
Step 11: Push node into stack
Step 12: End For loop
Step 13: Return the top element of stack
Step 14: Stop

3. Inorder Recursive Traversal

Step 1: Start
Step 2: If root ≠ NULL, then do Steps 3–5
Step 3: Call INORDER(root.l)
Step 4: Print root.data
Step 5: Call INORDER(root.r)
Step 6: Stop

4. Preorder Recursive Traversal

Step 1: Start
Step 2: If root ≠ NULL, then do Steps 3–5
Step 3: Print root.data
Step 4: Call PREORDER(root.l)
Step 5: Call PREORDER(root.r)
Step 6: Stop

5. Postorder Recursive Traversal
Step 1: Start
Step 2: If root ≠ NULL, then do Steps 3–5
Step 3: Call POSTORDER(root.l)
Step 4: Call POSTORDER(root.r)
Step 5: Print root.data
Step 6: Stop

6. Inorder Non-Recursive Traversal
Step 1: Start
Step 2: Create an empty stack st
Step 3: Set current = root
Step 4: While (current ≠ NULL OR stack is not empty), repeat Steps 5–10
Step 5: While (current ≠ NULL), repeat Steps 6–7
Step 6: Push current into stack
Step 7: Set current = current.l
Step 8: End inner while
Step 9: Pop top element from stack into temp
Step 10: Print temp.data
Step 11: Set current = temp.r
Step 12: End outer while
Step 13: Stop

7. Preorder Non-Recursive Traversal
Step 1: Start
Step 2: Create an empty stack st
Step 3: Push root into stack
Step 4: While stack is not empty, repeat Steps 5–10
Step 5: Pop top element into temp
Step 6: Print temp.data
Step 7: If temp.r ≠ NULL, then
Step 8: Push temp.r into stack
Step 9: If temp.l ≠ NULL, then
Step 10: Push temp.l into stack
Step 11: End while
Step 12: Stop

Flowcharts:

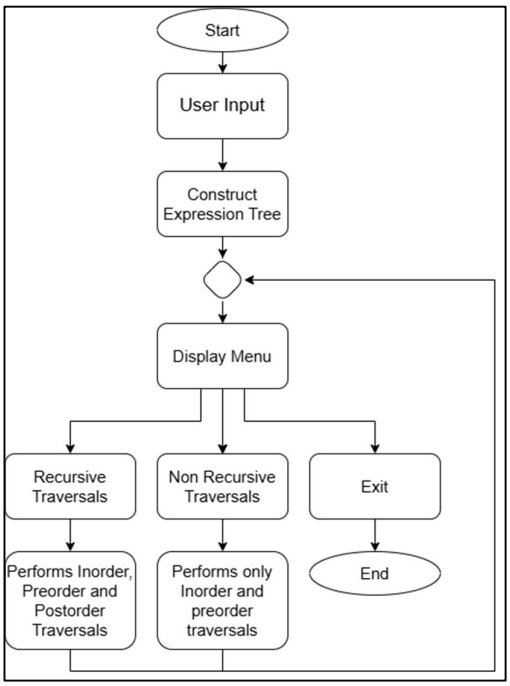

Fig 1.1 – Program Flow

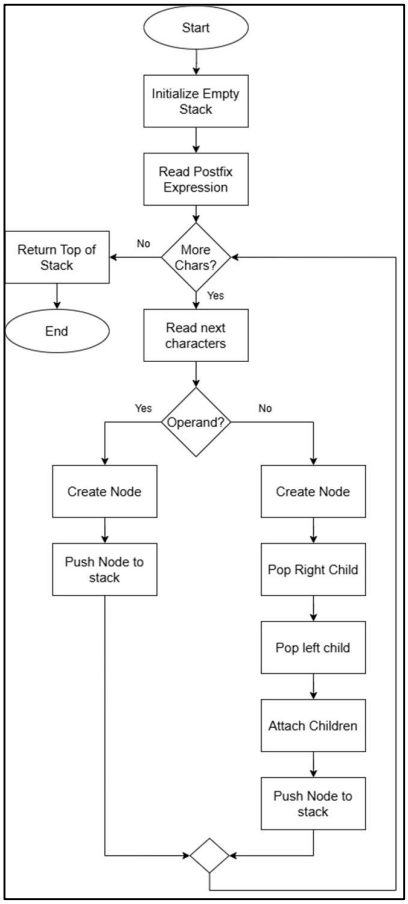

Fig 1.2 – Construct Tree

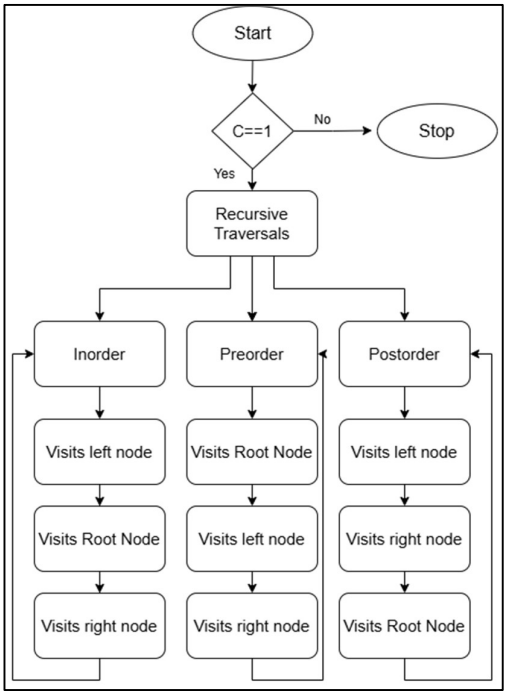

Fig 1.3 – Recursive Traversals

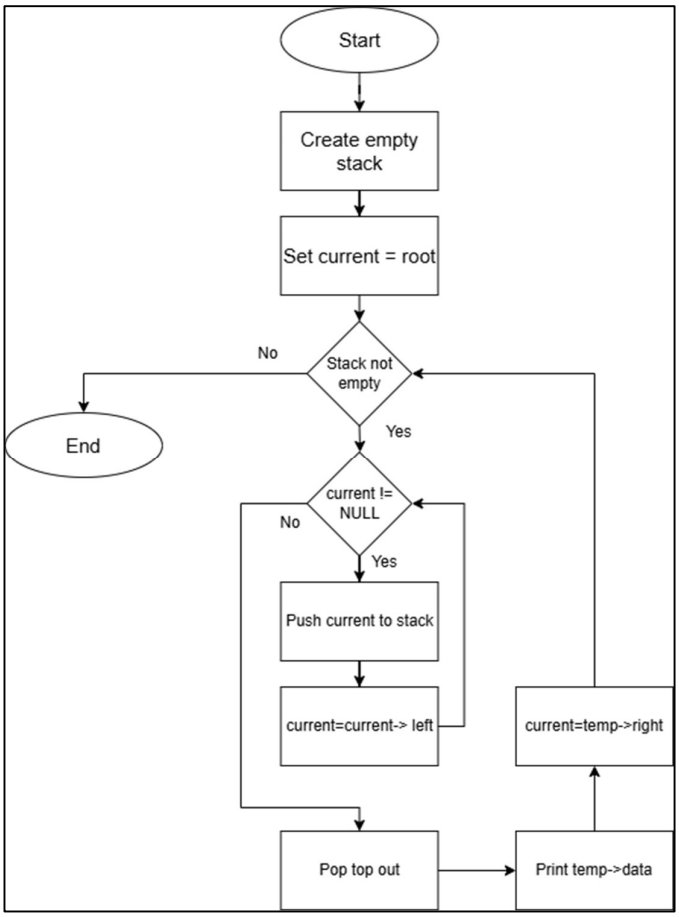

Fig 1.4 – Non-Recursive Inorder Traversal

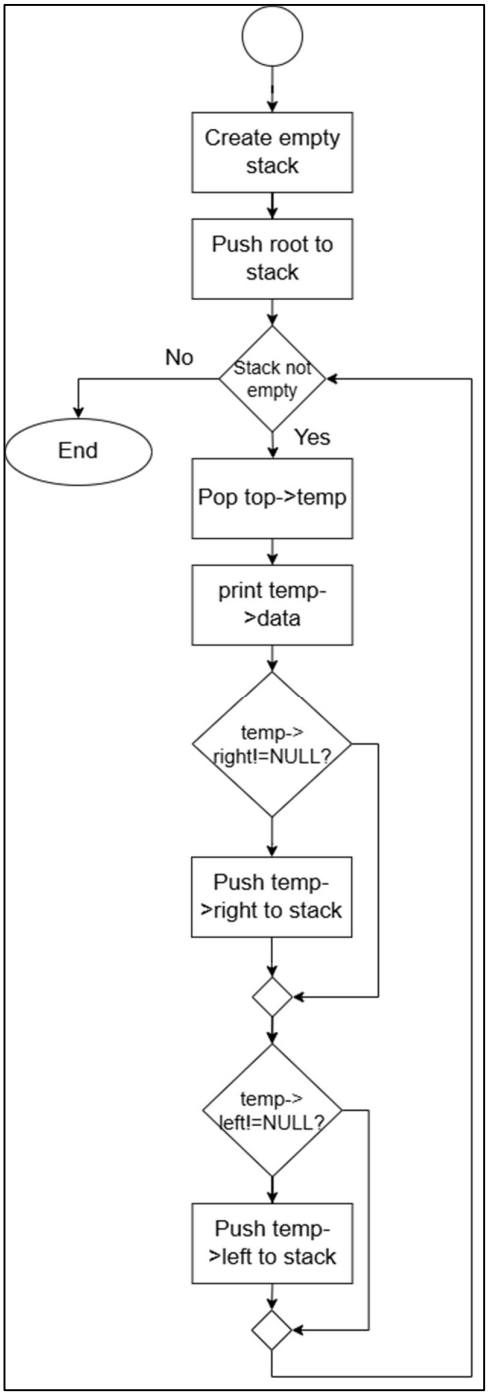

Fig 1.5 – Non-Recursive Preorder Traversal

Codes:

ExpressionTree.cpp

```
#include<iostream>
#include<stack>
#include <cctype>
using namespace std;

struct Node{
```

```
    char data;
    Node* l;
    Node* r;
};
Node* create(char data){
    Node* root=new Node();
    root->data=data;
    root->l=NULL;
    root->r=NULL;
    return root;
}
void inorder(Node* root){
    if(root!=NULL){
        inorder(root->l);
        cout<<root->data<<" ";
        inorder(root->r);
    }
}
void preorder(Node* root){
    if(root!=NULL){
        cout<<root->data<<" ";
        preorder(root->l);
        preorder(root->r);
    }
}
void postorder(Node* root){
    if(root!=NULL){
        postorder(root->l);
        postorder(root->r);
        cout<<root->data<<" ";
    }
}
Node* constructTree(string exp){
    stack<Node*> st;
    for(int i=0;i<exp.length();i++){
        if(isalnum(exp[i])){
            Node* root=create(exp[i]);
            st.push(root);
        }else{
            Node* root=create(exp[i]);
            root->r=st.top();
            st.pop();
            root->l=st.top();
            st.pop();
            st.push(root);
        }
    }
    return st.top();
}
void inorderNR(Node* root){
    stack<Node*> st;
    Node* a=root;
    while(a!=NULL || !st.empty()){
        while(a!=NULL){
            st.push(a);
            a=a->l;
        }
        Node* temp=st.top();
        st.pop();
```

```
        cout<<temp->data<<" ";
        a=temp->r;
    }
}
void preorderNR(Node* root){
    stack<Node*> st;
    st.push(root);
    while(!st.empty()){
        Node* temp=st.top();
        st.pop();
        cout<<temp->data<<" ";
        if(temp->r!=NULL){
            st.push(temp->r);
        }
        if(temp->l!=NULL){
            st.push(temp->l);
        }
    }
}
int main(){
    string exp;
    int c=0;
    cout<<"Enter postfix expression: "<<endl;
    cin>>exp;
    Node* root = constructTree(exp);
    while(c!=3){
    cout<<"Menu :"<<endl;
    cout<<"1. Recursive Traversals"<<endl;
    cout<<"2. Non-Recursive Traversals"<<endl;
    cout<<"Enter your choice: "<<endl;
    cin>>c;
    switch(c){
        case 1:
            cout<<"Recursive Traversals"<<endl;
            cout<<"Inorder: "<<endl;
            inorder(root);
            cout<<endl;
            cout<<"Preorder: "<<endl;
            preorder(root);
            cout<<endl;
            cout<<"Postorder: "<<endl;
            postorder(root);
            cout<<endl;
            break;
        case 2:
            cout<<"Non-Recursive Traversals"<<endl;
            cout<<"Inorder: "<<endl;
            inorderNR(root);
            cout<<endl;
            cout<<"Preorder: "<<endl;
            preorderNR(root);
            cout<<endl;
            break;
        case 3:
            cout<<"Bye Bye"<<endl;
            return 0;
        default:
            cout<<"Invalid Choice!"<<endl;
            break;
```

```
    }
    }
}
```

Outputs:

Expression Tree Traversals

```
Enter postfix expression:
ab*cd/+
Menu :
1. Recursive Traversals
2. Non-Recursive Traversals
Enter your choice:
1
Recursive Traversals
Inorder:
a * b + c / d
Preorder:
+ * a b / c d
Postorder:
a b * c d / +
```

```
Menu :
1. Recursive Traversals
2. Non-Recursive Traversals
Enter your choice:
2
Non-Recursive Traversals
Inorder:
a * b + c / d
Preorder:
+ * a b / c d
```